Lab 2

Selenium WebDriver cơ bản — Tương tác với trình duyệt và phần tử web

Hình thức nộp: Đường dẫn repository GitHub (public) — làm tiếp trên repo của Bài 1

---

1. Mục tiêu

Sau bài tập này, sinh viên có thể:

- Chọn được locator bền vững cho phần tử web và kiểm chứng bằng DevTools.
- Viết kịch bản đăng nhập cho cả luồng thuận và luồng nghịch.
- Thu thập dữ liệu từ danh sách phần tử và viết Assert có ý nghĩa.
- Dùng AI để tối ưu locator và tự thẩm định kết quả.
- Commit theo chuẩn Conventional Commits.

2. Yêu cầu

Phần A — Bảng locator (20%)

- Chọn **5 phần tử** trên saucedemo.com (ví dụ: ô Username, ô Password, nút Login, tên sản phẩm, giá sản phẩm).
- Với mỗi phần tử, đề xuất **ít nhất 2 cách định vị** khác nhau, nêu rõ chọn cách nào và **vì sao**.
- Kèm ảnh chụp DevTools dùng `Ctrl+F` chứng minh locator khớp `1 of 1`.
- Trình bày dạng bảng trong file `README.md`.

Phần B — Kịch bản đăng nhập: thuận & nghịch (30%)

- Ca 1 — luồng thuận: đăng nhập bằng `standard_user` / `secret_sauce`; Assert chuyển sang `inventory.html`.
- Ca 2 — luồng nghịch: đăng nhập sai (gợi ý: `locked_out_user` hoặc mật khẩu sai); Assert **thông báo lỗi hiện ra và đúng nội dung**.
- Dùng `[SetUp]` / `[TearDown]` đúng chuẩn; nhớ gọi `Quit()` và `Dispose()`.
- Chụp màn hình kết quả cả 2 ca đều **Passed**.

Phần C — Thu thập dữ liệu sản phẩm (25%)

- Sau khi đăng nhập, dùng `FindElements` lấy danh sách 6 sản phẩm; in ra tên và giá từng sản phẩm.

- Bắt buộc dùng `item.FindElement(...)` để lấy thông tin BÊN TRONG mỗi sản phẩm.
- Viết Assert **chặt hơn** việc chỉ đếm số lượng. Gợi ý: mọi giá đều bắt đầu bằng ký tự $; danh sách chứa đúng tên một sản phẩm cụ thể; không có tên nào rỗng.

Phần D — Lồng ghép AI (10%)

- Chọn 1 phần tử, copy `outerHTML` từ DevTools, nhờ AI đề xuất locator bền vững hơn.
- Ghi lại **prompt đã dùng** và câu trả lời của AI.
- Tự kiểm chứng locator AI đề xuất bằng `Ctrl+F`; viết nhận xét ngắn (3–5 câu): AI đúng/sai ở đâu?

Phần E — Quản lý mã nguồn (15%)

- Có **ít nhất 3 commit** theo chuẩn Conventional Commits (ví dụ: `test: them ca test dang nhap`).
- Push lên repository của Bài 1 (không tạo repo mới).

3. Hướng dẫn thực hiện sơ bộ

1. **Khảo sát trang trước**: mở saucedemo.com, chuột phải → Inspect từng phần tử. Ưu tiên tìm `id`, sau đó `data-test`, rồi mới tới CSS Selector.
2. **Kiểm chứng locator**: trong tab Elements bấm `Ctrl+F` (macOS: `Cmd+F`), dán locator, xác nhận kết quả `1 of 1` trước khi viết vào code.
3. **Tạo file test**: tạo `LoginTests.cs` trong project `FirstTests` của Bài 1. Khai báo `private IWebDriver driver;` ở cấp class.
4. **Viết SetUp/TearDown**: `[SetUp]` khởi tạo ChromeDriver + `GoToUrl`; `[TearDown]` gọi `Quit()` rồi `Dispose()`.
5. **Tách hàm dùng chung**: gom các bước đăng nhập vào một hàm phụ để ca test Phần C tái sử dụng — tránh lặp code.
6. **Chạy test**: `dotnet test`, hoặc dùng panel Testing của VS Code.
7. **Nếu gặp lỗi**: `NoSuchElementException` → kiểm tra lại locator bằng Ctrl+F. Sao chép NGUYÊN VĂN lỗi rồi hỏi AI kèm ngữ cảnh.
8. **Commit & push**: `git status` → `git add .` → `git commit -m "test: ..."` → `git push`.

4. Sản phẩm nộp

- Link repository GitHub (**public**) — chính là repo của Bài 1.

- File **`LoginTests.cs`** chứa các ca test của Phần B và C.
- File **`README.md`**: bảng locator (Phần A) và phần ghi nhận dùng AI (Phần D).
- Thư mục **`screenshots/`**: ảnh Ctrl+F, ảnh kết quả test Passed.

5. Tiêu chí chấm

Tiêu chí	Điểm	Ghi chú
Phần A — Bảng locator, có lý giải & ảnh kiểm chứng	**20%**	5 phần tử, ≥2 cách/phần tử
Phần B — Test đăng nhập thuận & nghịch, có Assert đúng	**30%**	Cả 2 ca đều Passed
Phần C — Thu thập dữ liệu, dùng item.FindElement, Assert chặt	**25%**	Không chỉ đếm số lượng
Phần D — Dùng AI tối ưu locator + tự thẩm định	**10%**	Có prompt & nhận xét
Phần E — Commit theo chuẩn Conventional Commits	**15%**	≥ 3 commit
Tổng cộng	**100%**	

Lưu ý: tuyệt đối KHÔNG dùng XPath tuyệt đối (bắt đầu bằng **`/html/body/...`**). Locator phải mô tả DANH TÍNH của phần tử, không mô tả đường đi.